

MoMo SMS Data Processing System

Database Design Document

Course: Database Systems — Week 2

Team: Divin Franco Manzi, Peace Maureen Umutesi, Alda Gatakokabasinga

Source data: modified_sms_v2.xml — 1,693 MTN MoMo SMS records

Database: MySQL 8.0+ — momo_sms_system

Contents

1. Entity Relationship Diagram (ERD)
2. Design Rationale
3. Data Dictionary
4. Sample Analytical Queries & Results
5. Security & Accuracy Rules

1. Overview

MoMo transaction processing app is a system that helps users to manage and organize their mobile money transaction. The system stores the user's transaction messages and then the design will store, analyze and organize data while maintaining integrity across the whole system to ensure reliability of the processes

1. Entity Relationship Diagram

Six entities: **Users**, **Transaction_Categories**, **Raw_SMS_Log**, **Transactions**, **Transaction_Participants** (M:N junction), and **System_Logs**.

Transaction_Categories	
PK category_id	INT
category_code	VARCHAR(30)
category_name	VARCHAR(60)
description	VARCHAR(255)
is_debit	BOOLEAN

1:M

Transactions	
PK transactions_id	INT
amount	DECIMAL(12,2)
transaction_time	DATETIME
processed_by	VARCHAR(30)
status	VARCHAR(20)
FK category_id	
fee	DECIMAL(12,2)
balance_after	DECIMAL(12,2)

Raw_SMS_Log	
PK sms_id.	INT
sender_address	VARCHAR(30)
raw_body	TEXT
sms_date	DATETIME
is_parsed	BOOLEAN

1:M

PK = Primary Key (bold+underlined) · FK = Foreign Key (bold) · Cardinality: 1:M = one-to-many · 1:1 = one-to-one

2. Design Rationale

Six entities, not four. The assignment requires Transactions, Users, Transaction_Categories, and System_Logs. We added two more because the raw XML data demanded it: **Raw_SMS_Log** and the junction table **Transaction_Participants**. Analysis of modified_sms_v2.xml (1,693 messages) showed roughly 60 are non-transactional (OTPs), so a raw SMS cannot always produce a transaction — this justified a separate Raw_SMS_Log table as an immutable staging area with an is_parsed flag: every message is stored verbatim, and only the messages that represent real transactions are promoted into Transactions, rather than force-fitting OTPs into the transaction table with null financial fields.

Why Transaction_Categories is a real table, not an ENUM column. Ten distinct transaction patterns emerged from the SMS bodies (incoming money, P2P transfer, merchant/code-holder payment, airtime, bundle purchase, bank deposit, agent withdrawal, linked-bank transfer, reversal, failed transaction). Storing this as a lookup table with a category_code and is_debit flag makes the categorization logic data-driven and lets new categories be added without a schema change or code redeploy.

Why the M:N relationship is Users↔Transactions via roles, not a padded one. Several transaction types in the data involve three parties, not two — agent withdrawals show a customer, an agent, and the transaction itself. Modeling sender_id / receiver_id as two FK columns cannot represent a third participant and breaks when a transaction has only one party (e.g., a bundle purchase debited from self). Instead, Transaction_Participants(transaction_id, user_id, participant_role) resolves the natural M:N: one user appears in many transactions, and one transaction involves multiple users in distinct roles (SENDER, RECEIVER, AGENT), matching real-world data rather than an artificial two-column shortcut.

System_Logs tracks data-processing events (parse success/failure per SMS) rather than application logs, keeping observability inside the database layer as the assignment specifies, linked back to the source SMS record for traceability.

3. Data Dictionary

Complete table and column descriptions for all six entities. All columns, types, NULLability, PK/FK, constraints and descriptions are documented below.

Users

Customers, agents and merchants referenced by MoMo transactions.

Column	Type	Constraints	Description
user_id	INT	NN, PK	Unique ID for a
full_name	VARCHAR(100)	NN	Name as it appears in the SMS body
phone_number	VARCHAR(15)	NN, UQ, CK(len≥6)	MSISDN, may be partially masked in source data
user_category	VARCHAR(20)	NN, DEF 'CUSTOMER', CK	chk_user_categoryRole this identity plays in MoMo ecosystem
created_at	DATETIME	NN, DEF NOW()	When this identity was first seen

person/agent/merchant found in the SMS

Transaction_Categories

Reference list of MoMo transaction types.

Column	Type	Constraints	Description
category_id	INT	NN, PK	Unique ID for a transaction category
category_code	VARCHAR(30)	NN, UQ	Machine-readable code used by the ETL p
category_name	VARCHAR(60)	NN	Human-readable label
description	VARCHAR(255)	NULL	What kind of SMS pattern maps to this category
is_debit	BOOLEAN	NN, DEF TRUE	TRUE if this category reduces account ba

Raw_SMS_Log

Immutable staging table holding every ingested SMS prior to parsing.

Column	Type	Constraints	Description
sms_id	INT	NN, PK	Unique ID for a raw SMS record
sender_address	VARCHAR(30)	NN	SMS sender address, e.g. M-Money
raw_body	TEXT	NN	Full untouched SMS body text
sms_date	DATETIME	NN	Timestamp the SMS was received
is_parsed	BOOLEAN	NN, DEF FALSE	TRUE once ETL has extracted a transacti

Transactions

Core fact table of confirmed/failed/reversed MoMo transactions.

Column	Type	Constraints	Description
processed_by	VARCHAR(30)	NULL	
status	VARCHAR(20)	NN, DEF 'COMPLETED', C	< chk_tx_sFinal stateatus
category_id	INT	NN, FK→Transaction_Cate	gories (RESTRICT)Category of this transaction
fee	DECIMAL(10,2)	NN, DEF 0, CK(≥0)	Fee charged for the transaction in RWF
balance_after	DECIMAL(12,2)	NULL	Account balance after the transaction
Column	Type	Constraints	Description
transaction_id	INT	NN, PK	Unique ID for a transaction
amount	DECIMAL(12,2)	NN, CK(>0)	Transaction amount in RWF
transaction_time	DATETIME	NN	Date/time the transaction was completed

Who processed it: SYSTEM, AGENT, or agent name from

Transaction_Participants

Junction table resolving Users ↔ Transactions (M:N) with roles.

Column	Type	Constraints	Description
participant_id	INT	NN, PK	Unique ID for a participation record
transaction_id	INT	NN, FK→Transactions (CASCADE)	Transaction this participation belongs to
user_id	INT	NN, FK→Users (RESTRICT)	User taking part in the transaction
participant_role	VARCHAR(20)	NN, UQ(tx,user,role), CK chk_participant_role	Role this user plays

System_Logs

ETL pipeline processing log for traceability and debugging.

Column	Type	Constraints	Description
log_id	INT	NN, PK	Unique ID for a log entry
related_sms_id	INT	NULL, FK→Raw_SMS_Log	(SET NULL)The SMS record being processed
process_step	VARCHAR(40)	NN	ETL stage, e.g. PARSE, CATEGORIZE, L
status	VARCHAR(10)	NN, CK chk_log_status	Outcome of this processing step
message	VARCHAR(255)	NULL	Detail message or error text
logged_at	DATETIME	NN, DEF NOW()	When this log entry was written

4. Sample Analytical Queries & Results

The queries below demonstrate analytical use of the schema with basic CRUD. All outputs come from the sample data in **database_setup.sql** (5 real transactions drawn from modified_sms_v2.xml).

5. Sample SQL QUERIES

Database changed

```
[mysql> SHOW TABLES;
```

Tables_in_momo_sms_system
Raw_SMS_Log
System_Logs
Transaction_Categories
Transaction_Participants
Transactions
Users
view_users_masked

7 rows in set (0.003 sec)

```
[mysql> SELECT * FROM users;
```

user_id	full_name	phone_number	user_category	created_at
1	Abebe Chala Chebudie	250795963036	CUSTOMER	2026-09-20
2	Jane Smith	250789013000	CUSTOMER	2026-09-20
3	Samuel Carter	250791666666	CUSTOMER	2026-09-20
4	Alex Doe	250790777777	CUSTOMER	2026-09-20
5	Robert Brown	250788999999	CUSTOMER	2026-09-20
6	Linda Green	250790777778	CUSTOMER	2026-09-20
7	Agent Sophia	250790777779	AGENT	2026-09-20
8	IREMBO Ltd	250700000001	MERCHANT	2026-09-20
9	ESICIA LTD KPAY	250700000002	MERCHANT	2026-09-20

9 rows in set (0.000 sec)

```
[mysql> SELECT * FROM transactions;
```

transaction_id	amount	transaction_time	processed_by	status
1	2000.00	2024-05-10 16:30:51	SYSTEM	COMPLETED
2	1000.00	2024-05-10 16:31:39	SYSTEM	COMPLETED
3	40000.00	2024-05-11 18:43:49	SYSTEM	COMPLETED
4	10000.00	2024-05-11 20:34:47	SYSTEM	COMPLETED
5	20000.00	2024-11-23 13:23:44	AGENT	COMPLETED

5 rows in set (0.001 sec)

Rules/constraints

1. Unique users: Match. user_id is PRIMARY KEY; uq_users_phone UNIQUE on phone_number.

2. Valid amounts (> 0): Match. `chk_tx_amount_positive` CHECK (amount > 0).
3. Valid status: SUCCESS/PENDING/FAILED: Different values. Our statuses are restricted by `chk_tx_status` to COMPLETED, FAILED, REVERSED — not SUCCESS/PENDING/FAILED.
4. Valid roles: SENDER/RECEIVER : Three roles, not two. `chk_participant_role` allows SENDER, RECEIVER, and AGENT (needed because a transaction can involve a customer + agent).
5. Valid log levels: INFO/WARNING/ERROR: No `log_level` concept. Our `System_Logs` has `process_step` (PARSE/CATEGORIZE/LOAD/DELETE_TRANSACTION) + status, restricted to SUCCESS/FAILED by `chk_log_status`.
6. Existing relationships : Partially. `fk_tx_category` (transactions→categories) ✓, `fk_participant_tx` (participants→transactions) ✓, `fk_participant_user` (participants→users) ✓, but logs link to `Raw_SMS_Log`, not transactions (`fk_log_sms` on `related_sms_id`) — a transaction deletion is audited via that SMS link.
7. No duplicate participation : Match. `uq_participant_tx_user` UNIQUE (transaction_id, user_id) — same user can't appear twice in one transaction.

Some of the constraints

- An invalid amount will be rejected.

```
mysql> INSERT INTO transactions (amount, transaction_time, processed_by,
[    -> VALUES (0, '2026-09-15 14:00:00', 'SYSTEM', 'COMPLETED', 1);
ERROR 3819 (HY000): Check constraint 'chk_tx_amount_positive' is violated.
```

- Invalid status will be rejected.

```
mysql> INSERT INTO transactions (amount, transaction_time, processed_by,
[    -> VALUES (1000.00, '2026-09-15 14:00:00', 'SYSTEM', 'INVALID', 1);
ERROR 3819 (HY000): Check constraint 'chk_tx_status' is violated.
```

- Invalid role will be rejected

```
mysql> INSERT INTO transaction_participants (transaction_id, user_id, participant_role,
[    -> VALUES (1, 3, 'INVALID');
ERROR 3819 (HY000): Check constraint 'chk_participant_role' is violated.
```

- Invalid log level will be rejected.

```
mysql> INSERT INTO system_logs (related_sms_id, process_step, status, message,
[    -> VALUES (1, 'PARSE', 'INVALID', 'Invalid log test');
ERROR 3819 (HY000): Check constraint 'chk_log_status' is violated.
```